



程序设计原理

分支结构

任课教师：喻 梅
于瑞国
王建荣
李雪威
赵满坤

Three Basic Control Structures of Programs

- All programs can be composed from these three structures.

1) Sequential · 顺序结构

Statements execute one after another in source order.

```
// Example
int a,b; cin >> a >> b;
int s = a + b;
cout << s << endl;
```

2) Selection / Branching · 分支结构

Choose a path based on condition(s) using `if/else` or `switch`.

```
// Example
int score; cin >> score;
if (score >= 60) cout << "Pass";
else cout << "Fail";
```

3) Looping / Iteration · 循环结构

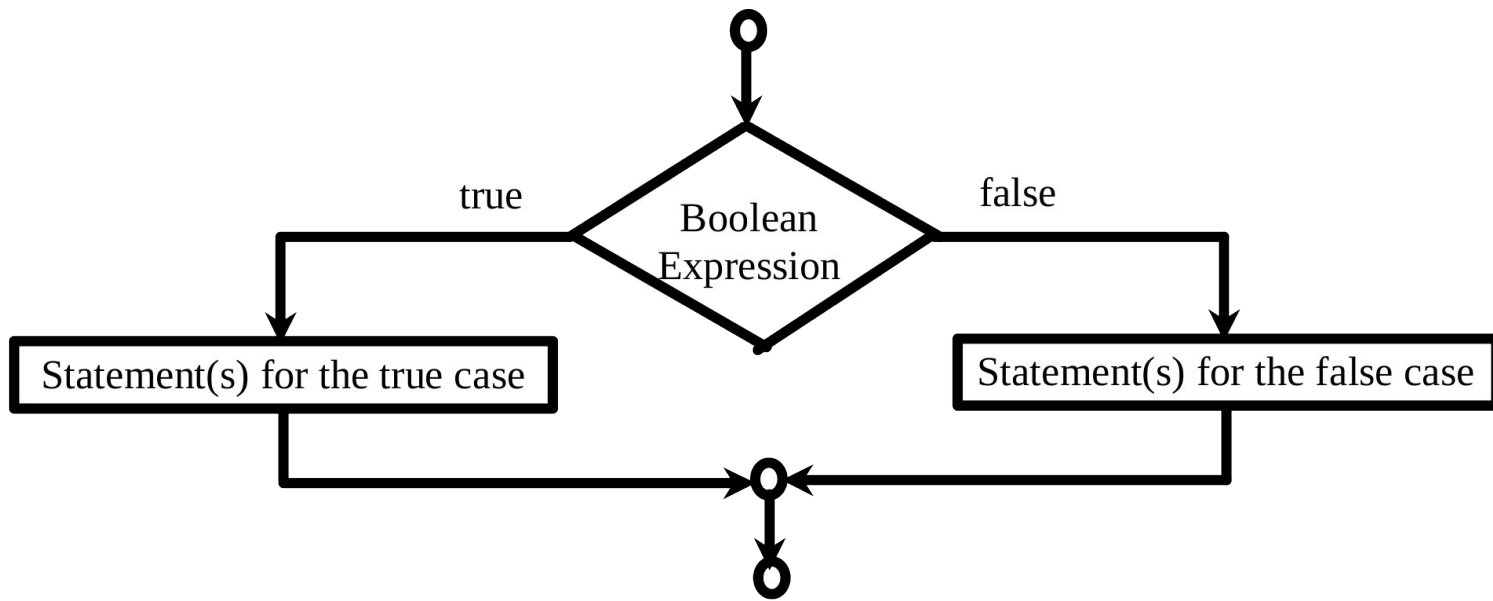
Repeat a block while a condition holds (or for a known count).

```
// Example
int n; cin >> n;
for (int i=1;i<=n;i++) cout << i << " ";
```

All complex programs are built by combining these three structures. Next: focus on **Selection (分支)**.

Selection: if-else Statements

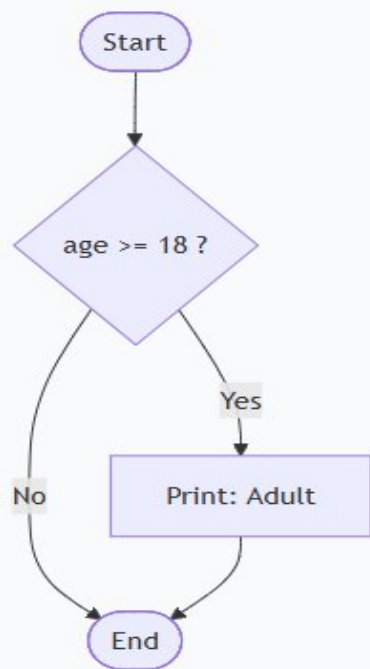
```
if (booleanExpression) {  
    statement(s)-for-the-true-case;  
}  
else {  
    statement(s)-for-the-false-case;  
}
```



if

Single-branch · 仅在条件为真时执行

```
// Print "Adult" only when age >= 18
int age; cin >> age;
if (age >= 18) {
    cout << "Adult" << endl;
}
```

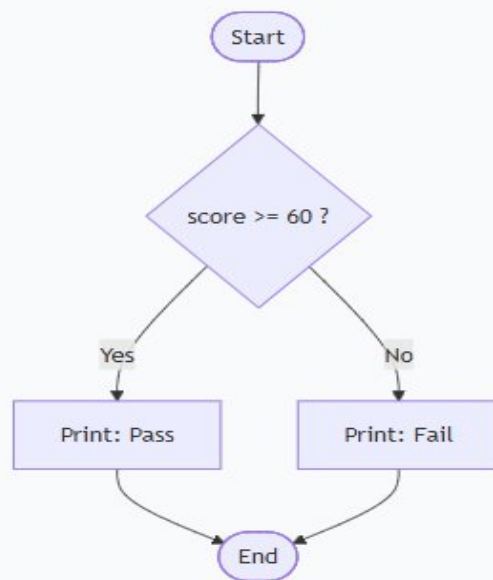


Use when the "false" case does nothing.

if-else

Two-way choice · 真走A，假走B

```
// Pass/Fail
int score; cin >> score;
if (score >= 60) {
    cout << "Pass" << endl;
} else {
    cout << "Fail" << endl;
}
```

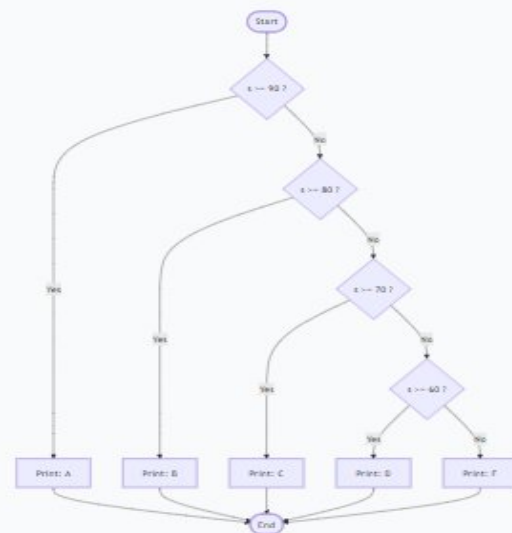


Use when both outcomes must be handled explicitly.

if-else if-else

Multi-way ladder · 多条件按顺序匹配一次

```
// Grade classification
int s; cin >> s;
if (s >= 90)      cout << "A";
else if (s >= 80) cout << "B";
else if (s >= 70) cout << "C";
else if (s >= 60) cout << "D";
else             cout << "F";
```



Conditions are checked top-down; the first match runs, the rest are skipped.

Selection: if Statements

```
#include <iostream>
using namespace std;
int main ()
{
    int m = 100;

    // check boolean expression
    if( m < 20 )    {
        // if the condition is true
        cout << "m 小于 20" << endl;
    }
    cout << "m 的值是 " << m << endl;
    return 0;
}
```

Selection: if-else Statements

```
#include <iostream>
using namespace std;
int main ()
{

    int m = 100;

    // check boolean expression
    if( m < 20 )    {
        //if the condition is true
        cout << "m 小于 20" << endl;
    }
    else    {
        // if the condition is false
        cout << "m 大于 20" << endl;
    }
    cout << "m 的值是 " << m << endl;
    return 0;
}
```

Selection: if - else if - else Statements

```
#include <iostream>
using namespace std;
int main()
{
    int a = 20;

    if (a<60){
        cout<<"Poor!"<<endl;
    }
    else if(a<70){
        cout<<"Fair!"<<endl;
    }
    else if(a<80){
        cout<<"Good!"<<endl;
    }
}
```

```
    else if(a<80){
        cout<<"Good!"<<endl;
    }
    else if(a<90){
        out<<"VeryGood!"<<endl;
    }
    else{
        cout<<"Excellent!"<<endl;
    }
    return 0;
}
```

Comparing and Testing Characters

```
if (ch >= 'A' && ch <= 'Z')  
    cout << ch << "is an uppercase letter" <<endl;  
else if (ch >='a' && ch <='z')  
    cout << ch << "is a lowercase letter" <<endl;  
else if (ch >= '0' && ch <= '9')  
    cout << ch << "is a numeric character" <<endl;
```


switch Statements

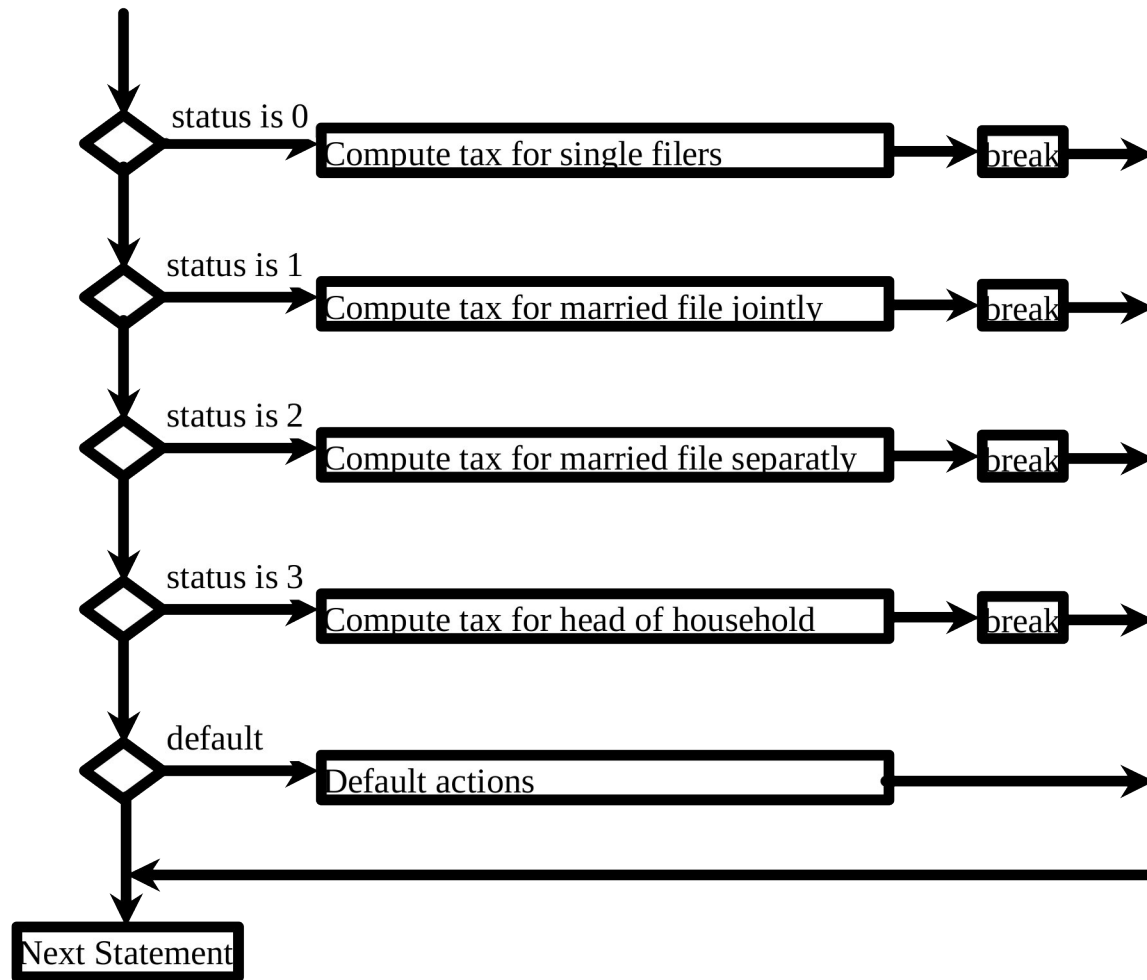
In real life, different people may be taxed at different rates depending on their **filing status**. We can use a **switch statement** to handle these categories more clearly than using many if statements.

Status	Filing Type / 纳税人类型	Tax Rate
0	Single 单身	10%
1	Married Jointly 已婚共同申报	12%
2	Married Separately 已婚单独申报	15%
3	Head of Household 家庭户主	20%

switch Statements

```
switch (status) {  
    case 0: compute taxes for single filers;  
            break;  
    case 1: compute taxes for married file jointly;  
            break;  
    case 2: compute taxes for married file separately;  
            break;  
    case 3: compute taxes for head of household;  
            break;  
    default: cout << "Error: invalid statud!";  
}
```

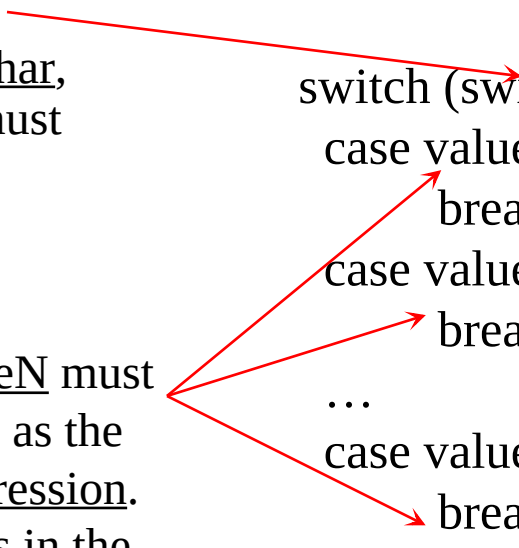
switch Statement Flow Chart



switch Statement Rules

The switch-expression must yield a value of char, short, or int type and must always be enclosed in parentheses.

The value1, ..., and valueN must have the same data type as the value of the switch-expression. The resulting statements in the case statement are executed when the value in the case statement matches the value of the switch-expression. Note that value1, ..., and valueN are constant expressions, meaning that they cannot contain variables in the expression, such as $1 + x$.



```
switch (switch-expression) {  
    case value1: statement(s)1;  
        break;  
    case value2: statement(s)2;  
        break;  
    ...  
    case valueN: statement(s)N;  
        break;  
    default: statement(s)-for-default;  
}
```

switch Statement Rules

The keyword break is optional, but it should be used at the end of each case in order to terminate the remainder of the switch statement. If the break statement is not present, the next case statement will be executed.

The default case, which is optional, can be used to perform actions when none of the specified cases matches the switch-expression.

```
switch (switch-expression) {  
    case value1: statement(s)1;  
        break;  
    case value2: statement(s)2;  
        break;  
    ...  
    case valueN: statement(s)N;  
        break;  
    default: statement(s)-for-default;  
}
```

The case statements are executed in sequential order, but the order of the cases (including the default case) does not matter. However, it is good programming style to follow the logical sequence of the cases and place the default case at the end.

Trace switch statement

```
char ch = 'a';  
switch (ch) {  
  case 'a': cout << ch;  
  case 'b': cout << ch;  
  case 'c': cout << ch;  
}
```

aaa

switch Statement

```
switch (ch) {  
    case 'a': cout << "aaaaaa"<<endl;  
                break;  
    default: cout<< "default!";  
    case 'b': cout << "bbbbbb"<<endl;  
                break;  
    case 'c': cout << "cccccc"<<endl;  
}
```

Summary

- Understand the purpose of branching: choose execution paths by conditions.
- Master the syntax and flow of `if` / `if-else` / `if-else if-else`.
- Use `switch` properly, including label matching and use cases.
- Understand *fall-through* and when to use `break`;
- Select appropriate control structures to solve real problems.

结构 / Structure	特点 / Feature	适用场景 / When to Use	注意事项 / Notes
<code>if</code>	单分支 / Single branch	只在条件为真时执行 / Do something only when condition is true	建议总写花括号 <code>{}</code> / Always use braces
<code>if - else</code>	双分支 / Two-way choice	必须处理两种结果 / Must handle both outcomes	注意边界与覆盖 / Watch boundaries and coverage
<code>if - else if - else</code>	多分支梯形 / Multi-way ladder	连续区间或多层条件 / Ranges or layered conditions	从高到低判断 / Order matters (top-down)
<code>switch</code>	多值匹配、结构清晰 / Discrete value matching	整型/字符/枚举的等值判断 / Equality on int/char/enum	每个 <code>case</code> 后写 <code>break</code> ; 理解 <i>fall-through</i> / Use <code>break</code> ; to avoid fall-through

Exercise

Small, Large, or Equal

Write a program which prints small/large/equal relation of given two integers a and b.

Input

Two integers a and b separated by a single space are given in a line.

Output

For given two integers a and b, print

$a < b$

if a is less than b,

$a > b$

if a is greater than b, and

$a == b$

if a equals to b.

Constraints

$-1000 \leq a, b \leq 1000$

Sample Input 1

1 2

Sample Output 1

$a < b$

Sample Input 2

4 3

Sample Output 2

$a > b$

Sample Input 3

5 5

Sample Output 3

$a == b$

Exercise

Range

Write a program which reads three integers a, b and c, and prints "Yes" if $a < b < c$, otherwise "No".

Input

Three integers a, b and c separated by a single space are given in a line.

Output

Print "Yes" or "No" in a line.

Constraints

$0 \leq a, b, c \leq 100$

Sample Input 1

1 3 8

Sample Output 1

Yes

Sample Input 2

3 8 1

Sample Output 2

No

Exercise

Sorting Three Numbers

Write a program which reads three integers, and prints them in ascending order.

Input

Three integers separated by a single space are given in a line.

Output

Print the given integers in ascending order in a line. Put a single space between two integers.

Constraints

$1 \leq \text{the three integers} \leq 10000$

Sample Input 1

3 8 1

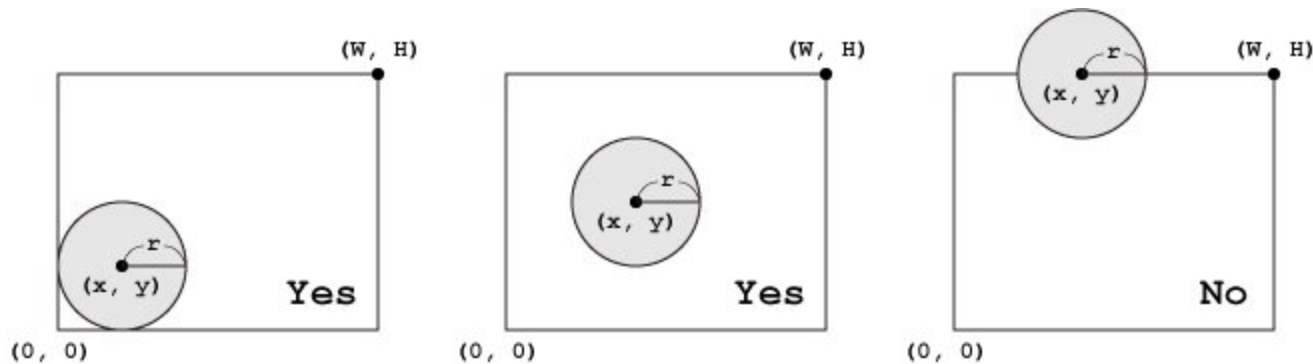
Sample Output 1

1 3 8

Exercise

Circle in a Rectangle

Write a program which reads a rectangle and a circle, and determines whether the circle is arranged inside the rectangle. As shown in the following figures, the upper right coordinate (W, H) of the rectangle and the central coordinate (x, y) and radius r of the circle are given.



Exercise

Input

Five integers W, H, x, y and r separated by a single space are given in a line.

Output

Print "Yes" if the circle is placed inside the rectangle, otherwise "No" in a line.

Constraints

$-100 \leq x, y \leq 100$ $0 < W, H, r \leq 100$

Sample Input 1

5 4 2 2 1

Sample Output 1

Yes

Sample Input 2

5 4 2 4 1

Sample Output 2

No